

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Resumen Teórico

APIs para Acceso a Bases de Datos — JDBC
Práctico N.º 4

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Arquitecturas de aplicaciones con acceso a base	2
3. ¿Qué es JDBC?	2
3.1. Arquitectura JDBC	3
4. Componentes principales de la API	3
5. URL JDBC	3
6. Carga del driver y conexión	4
7. Interfaz Connection	4
8. Tipos de Statement	5
9. Parámetros en la creación de Statement	5
10. Ejecución de sentencias	6
11. Interfaz ResultSet	6
12. Extracción de metadatos: DatabaseMetaData	7
13. Acceso al catálogo (cuando DatabaseMetaData no alcanza)	8
14. Correspondencia entre tipos SQL y Java	9
15. Persistencia de objetos en Java	9
16. Patrón típico con manejo de recursos	10
17. Conexión con el Práctico 4	10
18. Ideas clave para repasar antes del parcial	10
Resumen final	11

1. Introducción

Hasta este punto del cuatrimestre, todo lo que ejecutábamos sobre la base de datos lo hacíamos con un cliente como `psql`, `mysql` o `sqlplus`: un humano escribiendo SQL. El Práctico 4 cambia el ángulo: *programación cliente* de la base. Las aplicaciones reales no las escribe un humano comando a comando: las escribe un programa, casi siempre en un lenguaje de propósito general (Java, Python, C#, etc.) usando una **API de acceso a la base**.

El teórico se centra en la API de Java, **JDBC** (*Java DataBase Connectivity*), porque es la que se usa en el práctico, pero las ideas son generales: hace falta un *driver* específico del motor, una *conexión*, un objeto que represente la *sentencia* y una manera de *recorrer los resultados*.

2. Arquitecturas de aplicaciones con acceso a base

Hay dos arquitecturas básicas:

Dos capas La aplicación cliente se conecta *directamente* al servidor de base de datos. Es lo más simple y lo que se usa en los ejercicios del práctico: una aplicación de consola Java con JDBC contra MySQL/PostgreSQL/Oracle.

Tres capas La aplicación cliente se conecta a un *servidor de aplicaciones* y éste, a su vez, dialoga con la base. Se usa en sistemas más grandes (web, servicios) por motivos de seguridad, escalabilidad y reutilización de lógica.

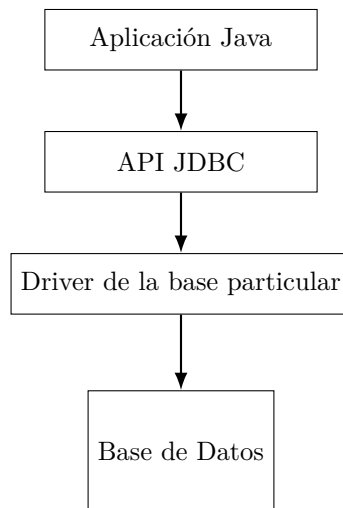
3. ¿Qué es JDBC?

JDBC (*Java DataBase Connectivity*) es una API Java que permite ejecutar operaciones sobre bases de datos relacionales, *independientemente del motor concreto*, usando SQL.

Características principales:

- Interacción con la base mediante SQL.
- 100 % Java (no requiere bindings nativos).
- Simple de usar: una decena de clases e interfaces alcanza.
- Alto rendimiento.
- Funciona contra cualquier base que provea un *driver JDBC*: MySQL, PostgreSQL, Oracle, Firebird, etc.

3.1. Arquitectura JDBC



La aplicación Java conoce *sólo* la API JDBC; el driver concreto traduce esa API al protocolo nativo del motor. Cambiar de motor cambia el driver y el URL, pero no las clases que usa la aplicación.

4. Componentes principales de la API

JDBC se construye alrededor de unos pocos componentes:

Componente	Rol
DriverManager (clase)	Carga el driver de la base elegida y crea conexiones.
Connection (interfaz)	Representa una sesión activa contra la base.
Statement y subclases (PreparedStatement, CallableStatement)	Representan una sentencia SQL para ejecutar.
ResultSet (interfaz)	Representa el conjunto de filas devueltas por una consulta.
DatabaseMetaData (interfaz)	Provee información de metadatos: tablas, columnas, claves, etc.

5. URL JDBC

Cada conexión JDBC se identifica con una URL del tipo:

```
jdbc:subprotocolo:fuentes
```

- Cada driver tiene su propio *subprotocolo*.
- Cada subprotocolo tiene su propia sintaxis para la *fuentes*.

Ejemplos.

```

jdbc:postgresql://host[:port]/database
jdbc:postgresql://localhost:5432/postgres

jdbc:mysql://host[:port]/database
jdbc:mysql://localhost:3306/practico1a_mysql

jdbc:oracle:thin:@//host:port/SID
jdbc:oracle:thin:@//localhost:1521/XE
  
```

6. Carga del driver y conexión

Clase DriverManager

Provee el método estático:

```
DriverManager.getConnection(String url, String user, String password);
```

Devuelve un objeto que implementa `Connection` y representa una sesión con la base. El *driver* debe estar previamente registrado; con `Class.forName(...)` se fuerza la carga explícita:

```
// Ejemplo: conexión a MySQL.
String driver = "com.mysql.cj.jdbc.Driver";
String url    = "jdbc:mysql://localhost/prueba";
String username = "root";
String password = "root";

// 1) Carga del driver (si todavía no está cargado).
Class.forName(driver);

// 2) Apertura de la conexión.
Connection connection =
    DriverManager.getConnection(url, username, password);
```

¿Es obligatorio `Class.forName`? Desde JDBC 4 los drivers se autorregistran vía `ServiceLoader`, así que en muchos casos basta con tener el JAR en el *classpath*. Aun así, el teórico y la mayoría de los ejemplos prácticos siguen usando `Class.forName` porque es la forma explícita y portable.

7. Interfaz Connection

Una `Connection` representa una sesión con la base. Permite:

- ejecutar instrucciones SQL y obtener resultados;
- obtener información de metadata (estructura) mediante `getMetaData()`;
- manejar transacciones con `commit()` y `rollback()`.

Métodos para crear sentencias

```
Statement      createStatement();
PreparedStatement prepareStatement(String sql);
CallableStatement prepareCall(String sql);
```

¿Por qué tres tipos? Por optimización y por casos de uso: el `Statement` es el más genérico, el `PreparedStatement` permite parámetros y precompilación, y el `CallableStatement` se usa para invocar procedimientos almacenados.

Transacciones: `setAutoCommit`

- Si `autoCommit` es *verdadero* (valor por defecto), *cada* sentencia ejecutada es una transacción independiente.
- Si se pasa a *falso*, hay que delimitar la transacción explícitamente con `commit()` o `rollback()`.

```
connection.setAutoCommit(false);
try {
    // varias sentencias...
    connection.commit();
} catch (SQLException e) {
    connection.rollback();
    throw e;
}
```

8. Tipos de Statement

Statement

Representa una sentencia SQL *estática*: no admite parámetros. Útil para sentencias armadas dinámicamente que no reciben datos del usuario, como las de exploración de metadatos.

PreparedStatement

Hereda de `Statement` y representa una sentencia *precompilada* con parámetros (placeholders ?). Ventajas:

- **Rendimiento**: el motor analiza y planifica una sola vez; las ejecuciones siguientes reutilizan el plan.
- **Seguridad**: como los parámetros viajan por separado de la sentencia, evita *SQL injection*.
- **Conversión de tipos**: el driver se encarga de convertir `int`, `String`, `Date`, etc. al tipo SQL correspondiente.

CallableStatement

Hereda de `Statement`; su objetivo es la *ejecución de procedimientos almacenados*. La sintaxis estándar (*escape call*) es:

```
{ ? = call funcion(?, ?) } // funcion con valor de retorno
{ call procedimiento(?, ?) } // procedimiento sin retorno
```

Para parámetros OUT o INOUT hay que registrarlos con `registerOutParameter(int pos, int sqlType)`.

9. Parámetros en la creación de Statement

`Connection` ofrece versiones de `createStatement` y `prepareStatement` que toman parámetros adicionales para controlar el `ResultSet` resultante.

Tipo de ResultSet

- `ResultSet.TYPE_FORWARD_ONLY` (default): el cursor sólo avanza hacia adelante.
- `ResultSet.TYPE_SCROLL_INSENSITIVE`: el cursor se mueve en ambos sentidos; *no* ve actualizaciones posteriores a la consulta.
- `ResultSet.TYPE_SCROLL_SENSITIVE`: ambos sentidos y refleja las actualizaciones cuando se producen.

Concurrencia / actualizabilidad

- `ResultSet.CONCUR_READ_ONLY` (default): sólo lectura.
- `ResultSet.CONCUR_UPDATABLE`: el resultado es actualizable desde el propio `ResultSet` (`updateXxx`, `insertRow`, `deleteRow`).

10. Ejecución de sentencias

Métodos principales de `Statement`:

- `ResultSet executeQuery(String sql)`: para `SELECT`; devuelve un `ResultSet`.
- `int executeUpdate(String sql)`: para `INSERT`, `UPDATE` o `DELETE`; devuelve la cantidad de filas afectadas.
- `boolean execute(String sql)`: para sentencias que pueden devolver múltiples resultados; `true` si el primer resultado es `ResultSet`.

11. Interfaz `ResultSet`

Un `ResultSet` provee acceso a la tabla generada por la ejecución de una consulta. Conceptualmente es un puntero (cursor) que apunta a una fila a la vez.

Recorrido y cierre

- `boolean next()`: avanza el cursor a la próxima fila. La *primera* llamada activa la primera fila. Devuelve `false` cuando se queda sin filas.
- `void close()`: libera el `ResultSet` (en la práctica, conviene usar `try-with-resources`).

Lectura de columnas

```
<Tipo> get<Tipo>(int columnIndex);    // por índice (empieza en 1)
<Tipo> get<Tipo>(String columnName);  // por nombre (menos eficiente)
```

Ejemplos.

```
int nro = rs.getInt("nro_cliente");
String nombre = rs.getString("nombre");
java.math.BigDecimal monto = rs.getBigDecimal("monto");
java.sql.Date fecha = rs.getDate("fecha");
```

Modificación de columnas (cursor actualizable)

```
update<Tipo>(String columnLabel, <Tipo> valor);
int findColumn(String columnName); // índice de la columna por nombre
```

Navegación (en `ResultSet` *scrollable*)

- `previous()`: una fila atrás.
- `first()`, `last()`: a la primera/última.
- `beforeFirst()`, `afterLast()`: antes de la primera o después de la última (sin posicionar en una fila válida).
- `relative(int n)`: salta *n* filas relativas a la actual.
- `absolute(int n)`: salta a la fila *n* absoluta.

Ejemplo completo

```
Statement statement = connection.createStatement();
String query = "SELECT * FROM persona";

// Envía el query a la base y almacena el resultado.
ResultSet resultSet = statement.executeQuery(query);

// Recorre los resultados.
while (resultSet.next()) {
    System.out.print(" DNI: " + resultSet.getString("DNI"));
    System.out.print("; Nombre: " + resultSet.getString("nombre"));
    System.out.print("; Email: " + resultSet.getString("email"));
    System.out.println();
}
```

12. Extracción de metadatos: DatabaseMetaData

Con `Connection.getMetaData()` se obtiene un objeto `DatabaseMetaData` que expone la *estructura* de la base. Es la pieza central del Ejercicio 3 del práctico.

Métodos más usados

`getTables.`

```
ResultSet getTables(String catalog, String schemaPattern,
                    String tableNamePattern, String[] types);
```

Devuelve un `ResultSet` con al menos las columnas:

- `TABLE_CAT` (catálogo, puede ser null).
- `TABLE_SCHEM` (esquema, puede ser null).
- `TABLE_NAME` (nombre).
- `TABLE_TYPE`: `TABLE`, `VIEW`, `SYSTEM TABLE`, `GLOBAL TEMPORARY`, etc.
- `REMARKS` (comentarios).

`getColumns.`

```
ResultSet getColumns(String catalog, String schemaPattern,
                    String tableNamePattern, String columnNamePattern);
```

Cada fila describe una columna; el `ResultSet` tiene al menos 20 columnas. Los códigos de tipo de dato son los definidos en `java.sql.Types`.

`getProcedures.`

```
ResultSet getProcedures(String catalog, String schemaPattern,
                        String procedureNamePattern);
```

Devuelve los procedimientos almacenados disponibles. Las constantes `procedureColumnIn`, `procedureColumnOut`, etc. del mismo `DatabaseMetaData` clasifican los parámetros.

Otros métodos útiles para el Ejercicio 3.

- `getPrimaryKeys(catalog, schema, table)`: devuelve las columnas de la PK con `KEY_SEQ`.
- `getIndexInfo(catalog, schema, table, unique, approximate)`: devuelve los índices; combinado con un filtro sobre los que coinciden con la PK, sirve para detectar *claves únicas adicionales*.

- `getImportedKeys(catalog, schema, table)` y `getExportedKeys(...)`: claves foráneas *salientes* o *entrantes*.

Ejemplo: listar esquemas en Oracle

```
String driver    = "oracle.jdbc.driver.OracleDriver";
String url      = "jdbc:oracle:thin:@//localhost:1521/XE";
String username = "USUARIO1";
String password = "USUARIO1";

Class.forName(driver);
Connection connection = DriverManager.getConnection(url, username, password);

DatabaseMetaData metaData = connection.getMetaData();
ResultSet resultSetSchemas = metaData.getSchemas();

System.out.println("Esquemas de la Base de datos");
while (resultSetSchemas.next()) {
    System.out.println("  esquema: " + resultSetSchemas.getString(1));
}
```

Ejemplo: listar tablas en PostgreSQL

```
String driver    = "org.postgresql.Driver";
String url      = "jdbc:postgresql://localhost:5432";
String username = "postgres";
String password = "root";

Class.forName(driver);
Connection connection = DriverManager.getConnection(url, username, password);

String[] tipo = { "TABLE" };
DatabaseMetaData metaData = connection.getMetaData();
ResultSet rs = metaData.getTables("Proyecto", "comparador", null, tipo);

while (rs.next()) {
    System.out.println("  catalogo: " + rs.getString(1));
    System.out.println("  esquema: " + rs.getString(2));
    System.out.println("  nombre: " + rs.getString(3));
    System.out.println("  tipo: " + rs.getString(4));
    System.out.println("  comentarios: " + rs.getString(5));
}
```

13. Acceso al catálogo (cuando DatabaseMetaData no alcanza)

Hay información que la API `DatabaseMetaData` no expone (detalles muy específicos de un motor). En esos casos se va *directamente al catálogo*: en MySQL existe la base `information_schema` y `mysql`; en PostgreSQL hay dos catálogos por base, ANSI y PostgreSQL; en Oracle se usan las vistas `USER_*`/`ALL_*`/`DBA_*`.

Ejemplo (MySQL). Listar nombres y código de los procedimientos del esquema `procedimientos`:

```
SELECT specific_name AS nombre, routine_definition AS codigo
FROM information_schema.ROUTINES R
WHERE routine_schema = 'procedimientos';
```

14. Correspondencia entre tipos SQL y Java

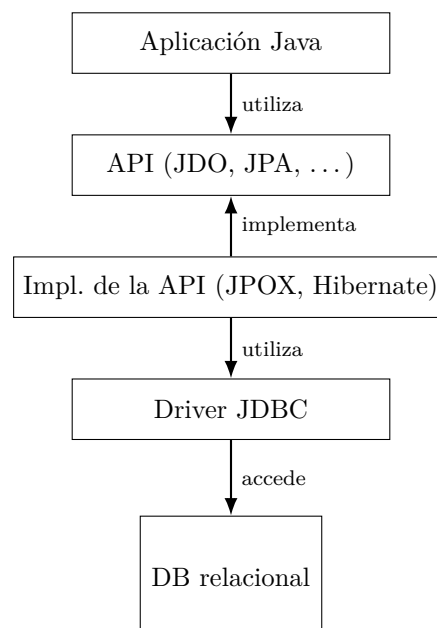
JDBC define una correspondencia estándar entre los tipos SQL y los tipos Java; los métodos `getXxx/setXxx` respetan esa tabla. Los datos que cruzan la red usan internamente `java.sql.Types` como contrato.

Tipo SQL	Tipo Java
CHAR, VARCHAR, LONGVARCHAR	String
NUMERIC, DECIMAL	java.math.BigDecimal
BIT	boolean
TINYINT	byte
SMALLINT	short
INTEGER	int
BIGINT	long
REAL	float
FLOAT, DOUBLE	double
BINARY, VARBINARY, LONGVARBINARY	byte[]
DATE	java.sql.Date
TIME	java.sql.Time
TIMESTAMP	java.sql.Timestamp

NUMERIC/DECIMAL → BigDecimal. Por eso en el Ejercicio 2 del práctico la cotización máxima se lee con `getBigDecimal(1)`: la columna SQL es `NUMERIC(12,4)`.

15. Persistencia de objetos en Java

Más allá de JDBC, la plataforma Java define APIs para *persistir objetos* en bases relacionales u objeto-relacionales: **JDO** (*Java Data Objects*), **JPA** (*Java Persistence API*), etc. Sus implementaciones (Hibernate, EclipseLink, JPOX, OpenJPA...) terminan apoyándose en JDBC para hablar con el motor.



16. Patrón típico con manejo de recursos

Los recursos JDBC son sensibles: si quedan abiertos pueden agotar el *pool* del motor. Desde Java 7, el patrón estándar es `try-with-resources`, que cierra automáticamente cualquier `AutoCloseable`.

```
String sql = "SELECT id, nombre FROM persona WHERE id = ?";

try (Connection connection = DriverManager.getConnection(url, user, pass);
    PreparedStatement preparedStatement = connection.prepareStatement(sql)) {

    preparedStatement.setInt(1, 42);

    try (ResultSet resultSet = preparedStatement.executeQuery()) {
        while (resultSet.next()) {
            int id = resultSet.getInt("id");
            String nombre = resultSet.getString("nombre");
            // ...
        }
    }
}
```

17. Conexión con el Práctico 4

- **Ejercicio 1:** usa `DriverManager`, `Connection`, `PreparedStatement` (con `setInt`/`setString`/`executeUpdate`/`executeQuery`) y `ResultSet` con `next()`/`getXxx`. Es el caso ABM clásico.
- **Ejercicio 2:** agrega `CallableStatement` para invocar la función `fn_actualizar_cotizacion_maxima` y leer un parámetro OUT con `registerOutParameter(..., Types.NUMERIC)` y `getBigDecimal(...)`. Suma un *fallback* con `SELECT` para entornos PostgreSQL donde el escape `{ ? = call ... }` no funciona.
- **Ejercicio 3:** usa `DatabaseMetaData` con `getTables`, `getColumns`, `getPrimaryKeys`, `getIndexInfo` y `getImportedKeys`. Es independiente del motor (PostgreSQL u Oracle) cambiando sólo el `.properties`.

18. Ideas clave para repasar antes del parcial

1. **JDBC es la API Java estándar de acceso a bases relacionales;** cada motor aporta su *driver* concreto.
2. **URL JDBC:** `jdbc:subprotocolo:fuente`. Conviene tenerla externalizada en un `.properties`.
3. **Pasos básicos de conexión:** `Class.forName + DriverManager.getConnection(url, user, pass)`.
4. **Connection** representa una sesión y es la fábrica de `Statement`, `PreparedStatement` y `CallableStatement`.
5. **Statement** (estático), *vs.* **PreparedStatement** (parametrizado, precompilado, seguro), *vs.* **CallableStatement** (procedimientos almacenados).
6. **Métodos de ejecución:** `executeQuery` (devuelve `ResultSet`), `executeUpdate` (devuelve `int`), `execute` (genérico).
7. **ResultSet** es un cursor sobre filas; `next()` lo avanza, `getXxx` lee columnas (por índice o por nombre).
8. **Tipos de ResultSet:** `FORWARD_ONLY` (default), `SCROLL_INSENSITIVE`, `SCROLL_SENSITIVE`. Concurrencia: `CONCUR_READ_ONLY` o `CONCUR_UPDATABLE`.
9. **Transacciones:** `setAutoCommit(false) + commit()/rollback()`; con `autoCommit true` cada sentencia es su propia transacción.

10. **Procedimientos almacenados:** *escape call { ? = call f(?) }*, parámetros IN con `setXxx`, OUT con `registerOutParameter`.
11. **DatabaseMetaData** expone tablas, columnas, PK, FK, índices, procedimientos, esquemas. Es la herramienta del Ejercicio 3.
12. **Correspondencia de tipos:** NUMERIC → BigDecimal, INTEGER → int, VARCHAR → String, DATE → java.sql.Date, TIMESTAMP → java.sql.Timestamp.
13. **try-with-resources** para Connection, Statement y ResultSet: nunca se debe olvidar el `close()`.
14. **Si DatabaseMetaData no alcanza**, se va al catálogo del motor (`information_schema` en MySQL/PostgreSQL, vistas `USER_*` en Oracle).
15. **Por encima de JDBC** viven las APIs de *persistencia de objetos* (JDO, JPA), pero su implementación sigue dialogando con la base por JDBC.

Resumen final

JDBC es la pieza que conecta el código de aplicación con la base. Su arquitectura es minimalista pero potente: un *driver* por motor, una *conexión* por sesión, distintos tipos de *statement* según el caso de uso, y un *ResultSet* para recorrer filas. **DatabaseMetaData** agrega una capa de introspección que es útil cuando hace falta conocer la estructura sin recurrir al catálogo del motor.

Si quedan firmes los cuatro bloques — *conexión*, *tipo de statement*, *recorrido de ResultSet* y *cierre con try-with-resources* —, el Práctico 4 se traduce directamente: cada uno de los tres ejercicios es una variación sobre un mismo esqueleto, intercambiando **PreparedStatement**, **CallableStatement** y **DatabaseMetaData** según lo que pida la consigna.